

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from tabulate import tabulate
4
5
6 def calculate_probability_table_sizes(vocab_size, sequence_length):
7     """
8     Calculate the size of probability tables for autoregressive language models
9     with different vocabulary sizes and sequence lengths.
10
11     Args:
12         vocab_size: Number of unique tokens in vocabulary
13         sequence_length: Maximum length of sequences
14
15     Returns:
16         Dictionary containing table sizes for each position n
17     """
18     table_sizes = {}
19     total_entries = 0
20
21     # Calculate table size for each position n in the sequence
22     for n in range(1, sequence_length + 1):
23         # For position n, we need  $V^n$  entries ( $V^{(n-1)}$  contexts *  $V$  possible next tokens)
24         entries = vocab_size ** n
25         table_sizes[f'p(x_{n}|x_1,...,x_{{n-1}})'] = entries
26         total_entries += entries
27
28     table_sizes['Total'] = total_entries
29     table_sizes['Formula'] = vocab_size * (1 - vocab_size ** sequence_length) / (1 -
vocab_size)
30
31     return table_sizes
32
33
34 def visualize_growth(vocab_sizes, max_sequence_length):
35     """
36     Visualize the exponential growth of probability table sizes
37
38     Args:
39         vocab_sizes: List of vocabulary sizes to compare
40         max_sequence_length: Maximum sequence length to consider
41     """
42     plt.figure(figsize=(12, 8))
43
44     for v in vocab_sizes:
45         sequence_lengths = range(1, max_sequence_length + 1)
46         table_sizes = [calculate_probability_table_sizes(v, n)['Total'] for n in
sequence_lengths]
47
48         plt.semilogy(sequence_lengths, table_sizes, marker='o', label=f'V = {v}')
49
50     plt.grid(True, which="both", ls="--")
51     plt.xlabel('Sequence Length (N)')

```

```

52 plt.ylabel('Number of Table Entries (log scale)')
53 plt.title('Exponential Growth of Probability Tables in Autoregressive
Language Models')
54 plt.legend()
55
56 return plt
57
58
59 # Example usage with a simple demonstration
60 def main():
61     print("DEMONSTRATING EXPONENTIAL GROWTH IN
AUTOREGRESSIVE LANGUAGE MODELS")
62     print("=" * 70)
63
64     # Parameters for demonstration
65     vocab_size = 5 # Small vocabulary for demonstration (e.g., A, B, C, D, E)
66     sequence_length = 8 # Maximum sequence length to consider
67
68     # Calculate table sizes
69     table_sizes = calculate_probability_table_sizes(vocab_size, sequence_length)
70
71     # Display results in a table
72     headers = ["Conditional Probability", "Number of Entries"]
73     rows = [(k, f'{v:,}') for k, v in table_sizes.items() if k != 'Formula']
74
75     print(f'For vocabulary size V = {vocab_size}:')
76     print(tabulate(rows, headers=headers, tablefmt="grid"))
77     print()
78     print(f'Note: Formula verification: {table_sizes['Total']:,} = {table_sizes['
Formula']:,0f}')
79     print("The formula is:  $V * (1 - V^N) / (1 - V)$ ")
80
81     # Now compare different vocabulary sizes
82     vocab_sizes = [2, 5, 10, 26] # Binary, small, medium, alphabet
83     plt = visualize_growth(vocab_sizes, sequence_length)
84
85     print("\nGenerating plot to visualize the exponential growth...")
86     plt.savefig('autoregressive_growth.png')
87     print("Plot saved as 'autoregressive_growth.png'")
88
89     # Show a practical example with a real language model
90     print("\nPRACTICAL EXAMPLE")
91     print("For GPT models with vocabulary size of ~50,000 tokens:")
92     real_vocab = 50000
93
94     practical_examples = [
95         ("For a 2-gram model (n=2)", real_vocab ** 2),
96         ("For a 3-gram model (n=3)", real_vocab ** 3),
97         ("For a context of 10 tokens", real_vocab ** 10),
98         ("For a context of 100 tokens", float('inf')) # This would be a huge number
99     ]
100
101     for desc, entries in practical_examples:

```

```

102     if entries == float('inf'):
103         print(f"{desc}: Too large to compute (more than atoms in the universe)")
104     else:
105         print(f"{desc}: {entries:,.0f} entries needed")
106
107     print("\nThis is why neural networks are used to parameterize these
distributions")
108     print("instead of explicit probability tables.")
109
110
111 if __name__ == "__main__":
112     main()
113
114
115 """
116
117 DEMONSTRATING EXPONENTIAL GROWTH IN AUTOREGRESSIVE
LANGUAGE MODELS
118 =====
=====
119 For vocabulary size V = 5:
120 +-----+-----+
121 | Conditional Probability | Number of Entries |
122 +=====+=====+
123 | p(x_1|x_1,...,x_{0})) | 5          |
124 +-----+-----+
125 | p(x_2|x_1,...,x_{1})) | 25         |
126 +-----+-----+
127 | p(x_3|x_1,...,x_{2})) | 125        |
128 +-----+-----+
129 | p(x_4|x_1,...,x_{3})) | 625        |
130 +-----+-----+
131 | p(x_5|x_1,...,x_{4})) | 3,125      |
132 +-----+-----+
133 | p(x_6|x_1,...,x_{5})) | 15,625     |
134 +-----+-----+
135 | p(x_7|x_1,...,x_{6})) | 78,125     |
136 +-----+-----+
137 | p(x_8|x_1,...,x_{7})) | 390,625    |
138 +-----+-----+
139 | Total                | 488,280    |
140 +-----+-----+
141
142 Note: Formula verification: 488,280 = 488,280
143 The formula is:  $V * (1 - V^N) / (1 - V)$ 
144
145 Generating plot to visualize the exponential growth...
146 Plot saved as 'autoregressive_growth.png'
147
148 PRACTICAL EXAMPLE
149 For GPT models with vocabulary size of ~50,000 tokens:
150 For a 2-gram model (n=2): 2,500,000,000 entries needed
151 For a 3-gram model (n=3): 125,000,000,000,000 entries needed

```

```
152 For a context of 10 tokens: 97,656,250,000,000,007,450,947,110,441,295,901,655,  
    239,229,440 entries needed  
153 For a context of 100 tokens: Too large to compute (more than atoms in the  
    universe)  
154  
155 This is why neural networks are used to parameterize these distributions  
156 instead of explicit probability tables.  
157  
158 Process finished with exit code 0  
159 """"
```